

Control Statements and Program Development

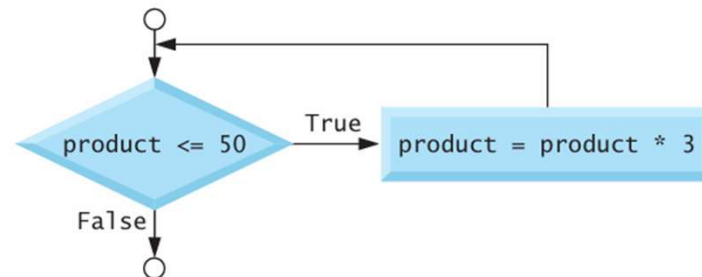
Lec-9_10_11_12

While

- The while statement allows you to repeat one or more actions while a condition remains True. Such a statement often is called a loop.
- Let's use a while statement to find the first power of 3 larger than 50:

```
In [1]: product = 3  
  
In [2]: while product <= 50:  
...:     product = product * 3  
...:  
  
In [3]: product  
Out[3]: 81
```

The following flowchart shows the preceding while statement's flow of control:

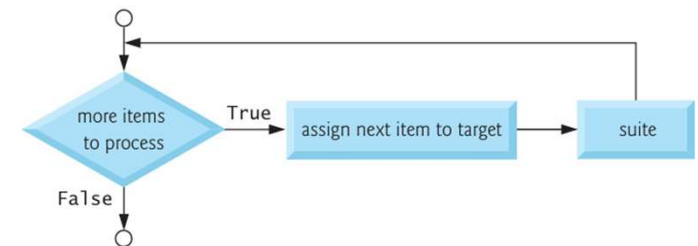


For

- Like the while statement, the for statement allows you to repeat an action or several actions. The for statement performs its action(s) for each item in a sequence of items. For example, a string is a sequence of individual characters. Let's display 'Programming' with its characters separated by two spaces:

```
In [1]: for character in 'Programming':  
...:     print(character, end='  ')  
...:  
P   r   o   g   r   a   m   m   i   n   g
```

The for statement's flowchart is similar to that of the while statement:



Function print's end Keyword Argument

- The built-in function print displays its argument(s), then moves the cursor to the next line. You can change this behavior with the argument end, as in
- `print(character, end=' ')`
- We used two spaces ('__'), so each call to print displays character's value followed by two spaces. So, all the characters display horizontally on the same line. Python calls end a keyword argument, but end is not a Python keyword.

Function print's sep Keyword Argument

- You can use the keyword argument `sep` (short for separator) to specify the string that appears between the items that `print` displays. When you do not specify this argument, `print` uses a space character by default. Let's display three numbers, each separated from the next by a comma and a space, rather than just a space.

```
In [2]: print(10, 20, 30, sep=', ')\n10, 20, 30
```

Iterables, Lists and Iterators

- The sequence to the right of the for statement's in keyword must be an iterable. An iterable is an object from which the for statement can take one item at a time until no more items remain. Python has other iterable sequence types besides strings. One of the most common is a list, which is a comma-separated collection of items enclosed in square brackets ([and]). The following code totals five integers in a list:

```
In [3]: total = 0

In [4]: for number in [2, -3, 0, 17, 9]:
...:     total = total + number
...:

In [5]: total
Out[5]: 25
```

Iterables, Lists and Iterators

- Each sequence has an iterator. The for statement uses the iterator “behind the scenes” to get each consecutive item until there are no more to process. The iterator is like a book mark—it always knows where it is in the sequence, so it can return the next item when it’s called upon to do so. For each number in the list, the suite adds the number to the total. When there are no more items to process, total contains the sum (25) of the list’s items. We cover lists in detail in the “Sequences: Lists and Tuples” chapter. There, you’ll see that the order of the items in a list matters and that a list’s items are mutable (that is, modifiable).

Built-In range Function

- Let's use a for statement and the built-in range function to iterate precisely 10 times, displaying the values from 0 through 9:

```
In [6]: for counter in range(10):  
...:     print(counter, end=' ')  
...:  
0 1 2 3 4 5 6 7 8 9
```

Augmented Assignments

- Augmented assignments abbreviate assignment expressions in which the same variable name appears on the left and right of the assignment's `=`, as `total` does in:

`Total = total + 5`

Augmented assignment	Sample expression	Explanation	Assigns
<i>Assume: c = 3, d = 5, e = 4, f = 2, g = 9, h = 12</i>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>**=</code>	<code>f **= 3</code>	<code>f = f ** 3</code>	8 to f
<code>/=</code>	<code>g /= 2</code>	<code>g = g / 2</code>	4.5 to g
<code>//=</code>	<code>g //= 2</code>	<code>g = g // 2</code>	4 to g
<code>%=</code>	<code>h %= 9</code>	<code>h = h % 9</code>	3 to h

Sequence Controlled Statements

- A requirements statement describes what a program is supposed to do, but not how the program should do it. Consider the following simple requirements statement and build a program:
A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

Execution Phases

- Execution Phases We used blank lines and comments to break the script into three execution phases—initialization, processing and termination
- The initialization phase creates the variables needed to process the grades and set these variables to appropriate initial values.
- The processing phase processes the grades, calculating the running total and counting the number of grades processed so far.
- The termination phase calculates and displays the class average.
- Q : What is definite repetition? Ans : If the number of repetitions is known before the loop begins executing.

Sentinel Controlled Repetition

- Q What is sentinel controlled repetition? What is a sentinel value?
- One way to solve this problem is to use a special value called a sentinel value (also called a signal value, a dummy value or a flag value) to indicate “end of data entry.” This is a bit like the way a caboose “marks” the end of a train. The user enters grades one at a time until all the grades have been entered. The user then enters the sentinel value to indicate that there are no more grades. Sentinel-controlled repetition is often called indefinite repetition because the number of repetitions is not known before the loop begins executing.

Sentinel Controlled Repetition

- A sentinel value must not be confused with any acceptable input value. Grades on a quiz are typically nonnegative integers between 0 and 100, so the value -1 is an acceptable sentinel value for this problem. Thus, a run of the class-average program might process a stream of inputs such as 95, 96, 75, 74, 89 and -1 . The program would then compute and print the class average for the grades 95, 96, 75, 74 and 89. The sentinel value -1 should not enter into the averaging calculation.
- What is divide and conquer?

Using Type Decimal for Monetary Amounts

- Try printing 1.20 – 1.18 upto 20 decimal places and see what happens.
- Bogus digits appear. This happens because after a certain number of digits the values are then approximated in binary and the bits are finite.
- The Python Standard Library provides many predefined capabilities you can use in your Python code to avoid “reinventing the wheel.” For monetary calculations and other applications that require precise representation and manipulation of numbers with decimal points, the Python Standard Library provides type Decimal, which uses a special coding scheme to solve the problem of to-the-penny precision. That scheme requires additional memory to hold the numbers and additional processing time to perform calculations but provides the to-the-penny precision required for monetary calculations. Banks also have to deal with other issues such as using a fair rounding algorithm when they’re calculating daily interest on accounts.

Importing Type Decimal from the decimal Module

- The Decimal type is not built into Python. Rather, it's part of the Python Standard Library, which is divided into modules—groups of related capabilities. The decimal module defines type Decimal and its capabilities.
- To use capabilities from a module, you must first import the entire module, as in `import decimal`
- and refer to the Decimal type as decimal.Decimal, or you must indicate a specific capability to import using from...import, as we do here

```
In [4]: from decimal import Decimal
```

Creating Decimals

- Decimals are created from strings.

```
In [5]: principal = Decimal('1000.00')
```

```
In [6]: principal
```

```
Out[6]: Decimal('1000.00')
```

```
In [7]: rate = Decimal('0.05')
```

```
In [8]: rate
```

```
Out[8]: Decimal('0.05')
```

```
In [9]: x = Decimal('10.5')
```

```
In [10]: y = Decimal('2')
```

```
In [11]: x + y
```

```
Out[11]: Decimal('12.5')
```

```
In [12]: x // y
```

```
Out[12]: Decimal('5')
```

```
In [13]: x += y
```

```
In [14]: x
```

```
Out[14]: Decimal('12.5')
```

- Decimal Arithmetic Decimals support the standard arithmetic operators `+`, `-`, `*`, `/`, `//`, `**` and `%`, as well as the corresponding augmented assignments

Break and Continue

- The break and continue statements alter a loop's flow of control. Executing a break statement in a while or for immediately exits that statement. In the following code, range produces the integer sequence 0–99, but the loop terminates when number is 10
- Executing a continue statement in a while or for loop skips the remainder of the loop's suite. In a while, the condition is then tested to determine whether the loop should continue executing. In a for, the loop processes the next item in the sequence (if any)

```
In [1]: for number in range(100):  
...:     if number == 10:  
...:         break  
...:     print(number, end=' ')  
...:  
0 1 2 3 4 5 6 7 8 9
```

```
In [2]: for number in range(10):  
...:     if number == 5:  
...:         continue  
...:     print(number, end=' ')  
...:  
0 1 2 3 4 6 7 8 9
```

Boolean Operators (And, or, not)

- The conditional operators `>`, `=`, `<=`, `==` and `!=` can be used to form simple conditions such as `grade >= 60`.

Boolean Operator and

To ensure that two conditions are *both* True before executing a control statement's suite, use the **Boolean and operator** to combine the conditions. The following code defines two variables, then tests a condition that's True if and only if *both* simple conditions are True—if either (or both) of the simple conditions is False, the entire and expression is False:

```
In [1]: gender = 'Female'

In [2]: age = 70

In [3]: if gender == 'Female' and age >= 65:
...:     print('Senior female')
...:
Senior female
```

The if statement has two simple conditions:

- `gender == 'Female'` determines whether a person is a female and
- `age >= 65` determines whether that person is a senior citizen.

The simple condition to the left of the and operator evaluates first because `==` has higher precedence than and. If necessary, the simple condition to the right of and evaluates next, because `>=` has higher precedence than and. (We'll discuss shortly why the right side of an and operator evaluates *only* if the left side is True.) The entire if statement condition is True if and only if *both* of the simple conditions are True. The combined condition can be made clearer by adding redundant (unnecessary) parentheses

```
(gender == 'Female') and (age >= 65)
```

<i>expression1</i>	<i>expression2</i>	<i>expression1 and expression2</i>
False	False	False
False	True	False
True	False	False
True	True	True

Boolean Operators (And, or, not)

Boolean Operator or

Use the **Boolean or operator** to test whether one *or* both of two conditions are True. The following code tests a condition that's True if either *or* both simple conditions are True—the entire condition is False only if *both* simple conditions are False:

```
In [4]: semester_average = 83
```

```
In [5]: final_exam = 95
```

```
In [6]: if semester_average >= 90 or final_exam >= 90:  
...:     print('Student gets an A')  
...:
```

```
Student gets an A
```

expression1	expression2	expression1 Or expression2
False	False	False
False	True	True
True	False	True
True	True	True

Snippet [6] also contains two simple conditions:

- `semester_average >= 90` determines whether a student's average was an A (90 or above) during the semester, and
- `final_exam >= 90` determines whether a student's final-exam grade was an A.

The truth table below summarizes the Boolean or operator. Operator and has higher precedence than or.

Boolean Operators (And, or, not)

The **Boolean not operator** “reverses” the meaning of a condition—True becomes False and False becomes True. This is a **unary operator**—it has only *one* operand. You place the not operator before a condition to choose a path of execution if the original condition (without the not operator) is False, such as in the following code:

```
In [7]: grade = 87

In [8]: if not grade == -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

Often, you can avoid using not by expressing the condition in a more “natural” or convenient manner. For example, the preceding if statement can also be written as follows:

```
In [9]: if grade != -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

The truth table below summarizes the not operator.

expression	not expression
False	True
True	False

Operators	Grouping
()	left to right
**	right to left
* / // %	left to right
+ -	left to right
< <= > >= == !=	left to right
not	left to right
and	left to right
or	left to right

Short Circuit Evaluation

- Python stops evaluating an and expression as soon as it knows whether the entire condition is False. Similarly, Python stops evaluating an or expression as soon as it knows whether the entire condition is True.
- This is called short-circuit evaluation. So the condition:
`gender == 'Female' and age >= 65`
- stops evaluating immediately if gender is not equal to 'Female' because the entire expression must be False. If gender is equal to 'Female', execution continues, because the entire expression will be True if the age is greater than or equal to 65.

Short Circuit Evaluation

- Similarly, the condition

`semester_average >= 90 or final_exam >= 90`

- stops evaluating immediately if `semester_average` is greater than or equal to 90 because the entire expression must be True. If `semester_average` is less than 90, execution continues, because the expression could still be True if the `final_exam` is greater than or equal to 90. In operator expressions that use `and`, make the condition that's more likely to be False the leftmost condition. In `or` operator expressions, make the condition that's more likely to be True the leftmost condition. These can reduce a program's execution time.

Intro to Data Science (Mean, Median, Mode)

- Here we continue our discussion of using statistics to analyze data with several additional descriptive statistics, including:
- mean—the average value in a set of values.
- median—the middle value when all the values are arranged in sorted order.
- mode—the most frequently occurring value.
- These are measures of central tendency—each is a way of producing a single value that represents a “central” value in a set of values, i.e., a value which is in some sense typical of the others.

```
In [3]: import statistics
```

Then, you can access the module’s functions with “statistics.” followed by the name of the function to call. The following calculates the grades list’s mean, median and mode, using the statistics module’s **mean**, **median** and **mode** functions:

```
In [4]: statistics.mean(grades)
Out[4]: 79.4
```

```
In [5]: statistics.median(grades)
Out[5]: 85
```

```
In [6]: statistics.mode(grades)
Out[6]: 85
```

Each function’s argument must be an *iterable*—in this case, the list `grades`. To confirm that the median and mode are correct, you can use the built-in **sorted** function to get a copy of `grades` with its values arranged in increasing order:

```
In [7]: sorted(grades)
Out[7]: [45, 85, 85, 89, 93]
```